

Assignment Code: DA-AG-007 Foundations of Machine Learning and EDA| Assignment

Question 1 : What is the difference between AI, ML, DL, and Data Science? Provide a brief explanation of each.

Ans— Artificial Intelligence (AI), Machine Learning (ML), Deep Learning (DL), and Data Science are interconnected fields, but each has a different scope, techniques, and applications. Artificial Intelligence (AI) is the broadest concept—it refers to machines designed to perform tasks that normally require human intelligence, such as reasoning, problem-solving, decision-making, speech recognition, and vision. AI uses rule-based systems, knowledge graphs, optimization algorithms, and ML models to make machines behave intelligently. Its applications include chatbots, autonomous vehicles, expert systems, robotics, and recommendation engines.

Machine Learning (ML) is a subfield of AI that focuses on enabling machines to learn patterns from data instead of being explicitly programmed. ML uses techniques like supervised learning (regression, classification), unsupervised learning (clustering, dimensionality reduction), and reinforcement learning to make predictions or decisions. ML powers spam detection, fraud detection, credit scoring, medical diagnosis tools, customer churn prediction, and product recommendations.

Deep Learning (DL) is an advanced subfield of ML based on artificial neural networks with many layers. It handles complex and high-dimensional data like images, videos, audio, and natural language. DL techniques include Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Transformers, and Generative Models. Applications include facial recognition, self-driving cars, speech-to-text, language translation, and advanced image/audio processing. DL requires large datasets and high computational power.

Data Science, on the other hand, is a broader field that focuses on extracting meaningful insights from structured and unstructured data by combining statistics, programming, ML, visualization, and domain knowledge. It involves data collection, cleaning, analysis, modeling, and communicating insights through dashboards or reports. Data Science uses tools like Python, SQL, R, Tableau, and ML algorithms to solve business problems such as sales forecasting, customer segmentation, market analysis, and operational optimization.

In simple terms we can say that AI is the broad field of creating intelligent systems, ML is a subset where machines learn from data, DL is a specialized ML approach using deep neural networks, and Data Science is the practice of analyzing data to extract insights and build data-driven solutions. AI is the broad goal of machine intelligence, ML is the data-driven method to achieve it, DL is the advanced neural-network-heavy branch of ML, and Data Science is the end-to-end process of extracting insights and building data-driven solutions.

Question 2: Explain overfitting and underfitting in ML. How can you detect and prevent them?

Ans- In Machine Learning, *overfitting* and *underfitting* are two major problems that affect a model's ability to generalize well to unseen data. Overfitting occurs when a model learns the training data too deeply, including noise and irrelevant patterns. This makes the model perform extremely well on training data but poorly on test data. It usually happens when the model is too complex, uses too many parameters, or is trained for too long. Examples include high training accuracy but low validation accuracy, or a model that reacts too strongly to minor fluctuations in data. Underfitting, on the other hand, happens when a model is too simple to capture the true patterns in the dataset. Here, both training and testing accuracy remain low because the model cannot learn the underlying relationships. This usually occurs due to insufficient training, overly simple algorithms (like using linear regression for a non-linear problem), or lack of important features.

A key concept for understanding these errors is the bias–variance tradeoff. Models with high bias are too simple and tend to underfit because they ignore important patterns. Models with high variance are too complex and tend to overfit because they memorize training data. A good ML model balances bias and variance to achieve optimal generalization. To detect overfitting or underfitting, we compare training vs. validation/test performance. Overfitting shows a large gap (training high, validation low), while underfitting shows uniformly low performance on both datasets. Visualization tools like learning curves further help identify these issues by showing how model performance changes with more training data.

Several techniques can prevent overfitting and underfitting. Cross-validation, especially K-Fold Cross-Validation, ensures that the model is tested on multiple subsets of data, helping detect instability and reducing the risk of overfitting. Regularization techniques such as L1 (Lasso), L2 (Ridge), and Elastic Net add penalties to overly complex models, thereby reducing variance. In deep learning, methods like dropout, batch normalization, and early stopping are commonly used. Reducing model complexity (fewer layers or parameters), pruning decision trees, or using simpler algorithms can also prevent overfitting. To prevent underfitting, we can add more relevant features, use more complex models, increase training epochs, or reduce regularization strength.

Data-related solutions also help manage both problems. For overfitting, collecting more data, data augmentation (especially for images), or removing noisy/irrelevant features improves generalization. For underfitting, ensuring proper feature engineering, using polynomial features for non-linear data, and improving data quality are useful strategies. Ultimately, the goal is to train a model that performs well not only on training data but also on unseen test data by achieving the right balance between bias and variance.

In simple term we can say Overfitting = model is too complex and memorizes data;
Underfitting = model is too simple and fails to learn patterns.

The bias–variance tradeoff represents the balance between two types of model errors: bias, which occurs when a model oversimplifies the data and leads to underfitting (such as using a linear model for a nonlinear pattern), and variance, which arises when a model becomes overly sensitive to small fluctuations in the training data, resulting in

overfitting (like an overly deep decision tree memorizing noise). Increasing model complexity reduces bias but increases variance, while simplifying the model increases bias but reduces variance, and the goal is to find an optimal point that minimizes total error. Cross-validation helps evaluate how well a model generalizes to unseen data by preventing overfitting and ensuring consistent performance across different train–test splits; the most common method, k-fold cross-validation, divides data into k segments, trains on k–1 folds, and tests on the remaining one, repeating this process k times and averaging the performance. Variants include Leave-One-Out CV, where each sample is used once as a test set, and Stratified k-Fold, which preserves class proportions for classification tasks. Regularization further combats overfitting by penalizing model complexity through techniques such as L1 regularization (Lasso), which encourages sparsity by pushing some weights to zero; L2 regularization (Ridge), which shrinks weights without removing them; and Elastic Net, which blends both L1 and L2 to handle correlated features. In neural networks, dropout reduces co-adaptation by randomly deactivating neurons during training, while early stopping halts training when validation error begins to increase, preventing the model from overfitting.

Question 3:How would you handle missing values in a dataset?

Explain at least three methods with examples.

Hint: Consider deletion, mean/median imputation, and predictive modeling.

Ans– Missing values are a common issue in real-world datasets and must be handled carefully to ensure accurate analysis and model performance. The method chosen depends on the amount of missing data, the type of variable (numeric or categorical), and the importance of the feature. Three widely used approaches are deletion methods, simple statistical imputation, and predictive modeling–based imputation.

1. Deletion Methods (Removing Missing Data) Deletion is useful when the percentage of missing values is small and randomly distributed.

- Listwise deletion removes entire rows containing any missing value. For example, if a dataset of employees has columns like Age, Salary, and Experience, and one record has Salary missing, the entire row is dropped.
- Pairwise deletion uses available data for computations without removing whole rows. This method is simple and maintains data integrity, but should be avoided when missing data is large or not random.

2. Mean/Median/Mode Imputation (Simple Statistical Imputation) This method replaces missing values with a central tendency metric.

- Mean imputation is used for normally distributed numeric data. Example: If the Age column has values [23, 25, NaN, 27], the missing value can be replaced by the mean age = $(23+25+27)/3 = 25$.
- Median imputation is preferred for skewed numeric data to avoid distortion. Example: For Salary values [30,000, 32,000, 90,000, NaN], replacing with the median (31,000–32,000 range) prevents the outlier (90,000) from affecting the result.
- Mode imputation is used for categorical data. Example: If “Department” has missing values, replacing them with the most frequent department (e.g., “Sales”) is a practical approach. This method is quick and preserves dataset size but may reduce variability.

3. Predictive Modeling (Advanced Imputation) When missing data is significant or patterns are complex, predictive models can estimate the missing values. The idea is to treat the missing-value column as a target variable and use other features to predict it.

- For numeric missing values, regression models such as Linear Regression, Random Forest Regressor, or KNN Imputer can be used. Example: Predicting missing Salary based on Age, Education, and Job Role.
- For categorical missing values, classification models such as Logistic Regression, Decision Trees, or Random Forest Classifier can be applied. Example: Predicting missing Department values using features like Experience, Role, and Location. This approach preserves data patterns and increases model accuracy but is computationally heavier and requires proper validation.

We can also say that Missing values can be handled using several methods. **1) Deletion:** Remove rows or columns with missing data when the missing portion is small. For example, if only 2 out of 1,000 rows have missing age values, you can drop those rows without affecting the dataset. **2) Mean/Median Imputation:** Replace missing numerical values with the mean or median of the column. For instance, if the "Salary" column has missing entries, you can fill them with the column's median to reduce the effect of outliers. **3) Predictive Modeling:** Use other features to predict the missing values. For example, if "Age" is missing, you can train a regression model using features like income, education, and occupation to estimate the missing ages.

Question 4: What is an imbalanced dataset? Describe two techniques to handle it (theoretical + practical). Hint: Discuss SMOTE, Random Under/Oversampling, and class weights in models.

Ans- An imbalanced dataset is one where the classes are not evenly represented—for example, 98% non-fraud cases and only 2% fraud cases. This imbalance causes models to focus on the majority class and ignore the minority class.

1) SMOTE (Synthetic Minority Oversampling Technique)

- Theory: Instead of duplicating minority samples, SMOTE creates synthetic new samples by interpolating between existing minority-class neighbors.
- Practical: Using `SMOTE()` from `imblearn`, you can generate synthetic fraud cases so the model learns minority patterns better.

2) Random Over/Undersampling

- Theory:
 - *Oversampling*: Increase minority class by duplicating samples.
 - *Undersampling*: Reduce majority class by removing samples.
- Practical:
 - Apply `RandomOverSampler()` to add more minority samples (e.g., more fraud cases).
 - Apply `RandomUnderSampler()` to reduce majority samples (e.g., reduce normal transactions).
You can also use class weights (e.g., in Logistic Regression), where the model gives higher penalty for misclassifying minority samples.

1) SMOTE (Oversampling with Synthetic Samples)

Theory:

SMOTE creates new synthetic samples for the minority class by interpolating between existing minority points.

Practical Example (Python):

```
from imblearn.over_sampling import SMOTE
```

```
sm = SMOTE()
X_res, y_res = sm.fit_resample(X, y)
```

If your dataset had 950 normal / 50 fraud, SMOTE might make it 950 normal / 950 fraud.

2) Random Oversampling / Undersampling

Theory:

- Oversampling: Duplicate minority samples.
- Undersampling: Remove majority samples.

Practical Example (Python):

Oversampling

```
from imblearn.over_sampling import RandomOverSampler

ros = RandomOverSampler()
X_res, y_res = ros.fit_resample(X, y)
```

Undersampling

```
from imblearn.under_sampling import RandomUnderSampler

rus = RandomUnderSampler()
X_res, y_res = rus.fit_resample(X, y)
```

3) Class Weights in Models

Theory:

Give higher weight to the minority class so the model penalizes mistakes more.

Practical Example:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(class_weight='balanced')
model.fit(X, y)
```

Question 5: Why is feature scaling important in ML? Compare Min-Max scaling and Standardization. Hint: Explain impact on distance-based algorithms (e.g., KNN, SVM) and gradient descent.

Ans- Importance of Feature Scaling in Machine Learning

Feature scaling is a crucial preprocessing step in Machine Learning because datasets often contain features with different units, ranges, and magnitudes. Algorithms that depend on distance, gradient optimization, or similarity measures can become biased toward larger-valued features if scaling is not applied. For example, if a dataset contains Age (0–100) and Salary (10,000–200,000), models may treat Salary as more important simply because of its larger numeric range, not because it truly has more predictive power.

Feature scaling is especially important for distance-based algorithms such as K-Nearest Neighbors (KNN), K-Means clustering, and Support Vector Machines (SVM), because they compute distances between points. Without scaling, features with large values dominate distance calculations, resulting in incorrect clustering or misclassification. Scaling is also critical for gradient-based algorithms like Linear Regression, Logistic Regression, and Neural Networks, because unscaled features cause gradients to update unevenly, slowing down convergence and making training unstable.

Min-Max Scaling vs. Standardization (Z-score Scaling)

1. Min-Max Scaling (Normalization)

Min-Max Scaling transforms features to a fixed range, usually 0 to 1:
$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- Impact: Keeps the original distribution shape but compresses values into the [0,1] range.
- Useful For: Algorithms sensitive to magnitude and where bounded values are needed, e.g., Neural Networks, KNN, K-Means, and distance-based models.
- Limitation: Sensitive to outliers—one extreme value can distort the scale.

Example: If Height ranges from 150–190 cm, a height of 160 becomes:
$$\frac{160 - 150}{190 - 150} = 0.25$$

2. Standardization (Z-score Scaling)

Standardization transforms data to have mean = 0 and standard deviation = 1:
$$X_{\text{standardized}} = \frac{X - \mu}{\sigma}$$

- Impact: Centers data at zero and normalizes the spread.
- Useful For: Algorithms assuming normally distributed data or using linear boundaries, e.g., Logistic Regression, SVM, Linear Regression, PCA, Gradient Descent-based models.
- Advantage: Less sensitive to outliers compared to Min-Max scaling because it spreads the data instead of compressing it.

Example: If mean Salary = 50,000 and std = 10,000, a salary of 60,000 becomes:
$$\frac{60,000 - 50,000}{10,000} = 1.0$$

Distance-based algorithms like KNN and SVM are highly sensitive to missing values, scaling, and class imbalance. KNN relies on distance measures such as Euclidean or Manhattan distance, so missing or unscaled data can distort these distances, leading to incorrect nearest neighbors and wrong predictions; additionally, in imbalanced datasets, KNN becomes biased toward the majority class because most neighbors belong to that class. Similarly, SVM depends on distance-based margins and kernel similarity, so missing or unscaled features can shift the separating hyperplane incorrectly, and class imbalance causes the margin to lean toward the minority class, often leading to misclassification due to fewer support vectors from that class. Gradient Descent is also impacted because missing values (NaNs) prevent proper gradient computation and can cause training failure, while unscaled features create uneven gradient magnitudes that lead to oscillation or very slow learning, harming convergence. In imbalanced data, the loss is dominated by the majority class, causing gradients to update in its favor, resulting in a model that predicts the majority class well but performs poorly on the minority class, especially in recall.

Question 6: Compare Label Encoding and One-Hot Encoding. When would you prefer one over the other? Hint: Consider categorical variables with ordinal vs. nominal relationships.

Ans- Here is the entire explanation rewritten in one clear paragraph:

Encoding categorical data is crucial because most machine learning algorithms require numerical input, and two common techniques are Label Encoding and One-Hot Encoding, which differ in how they convert categories into numbers and in the types of variables they suit. Label Encoding assigns each category a numerical value (e.g., Size = {Small, Medium, Large} → {0,1,2}), making it simple and memory-efficient, suitable for ordinal data with natural order, and effective for tree-based models where numeric order doesn't affect splits; however, it imposes an artificial ranking on nominal variables, causing algorithms like Linear Regression, Logistic Regression, SVM, and KNN to misinterpret encoded values as meaningful ranks. Label Encoding is best for features with logical ordering such as size levels or education levels. In contrast, One-Hot Encoding converts each category into separate binary columns (e.g., Male → [1,0], Female → [0,1]), making it ideal for nominal variables because it introduces no order and keeps categories independent, though it increases dimensionality and memory usage for high-cardinality features. One-Hot Encoding is preferred for models like logistic regression, linear models, SVM, KNN, and neural networks that require non-ordered numerical inputs.

Question 7: Google Play Store Dataset a). Analyze the relationship between app categories and ratings. Which categories have the highest/lowest average ratings, and what could be the possible reasons? Dataset: <https://github.com/MasteriNeuron/datasets.git> (Include your Python code and output in the code box below.)

Ans- `import pandas as pd`

```
df = pd.read_csv("C:/Users/khush/Downloads/googleplaystore.csv")
```

```
df = df.dropna(subset=['Rating'])
```

```
df['Rating'] = pd.to_numeric(df['Rating'], errors='coerce')
```

```
df = df.dropna(subset=['Rating'])
```

```
cat_avg = df.groupby("Category")["Rating"].mean().sort_values(ascending=False)
```

```
cat_avg
```

Category

1.9	19.000000
-----	-----------

EVENTS	4.435556
--------	----------

EDUCATION	4.389032
-----------	----------

ART_AND_DESIGN	4.358065
----------------	----------

BOOKS_AND_REFERENCE	4.346067
---------------------	----------

PERSONALIZATION	4.335987
-----------------	----------

PARENTING	4.300000
-----------	----------

GAME	4.286326
------	----------

BEAUTY	4.278571
--------	----------

HEALTH_AND_FITNESS	4.277104
--------------------	----------

SHOPPING	4.259664
----------	----------

SOCIAL	4.255598
--------	----------

WEATHER	4.244000
---------	----------

SPORTS	4.223511
--------	----------

PRODUCTIVITY	4.211396
HOUSE_AND_HOME	4.197368
FAMILY	4.192272
PHOTOGRAPHY	4.192114
AUTO_AND_VEHICLES	4.190411
MEDICAL	4.189143
LIBRARIES_AND_DEMO	4.178462
FOOD_AND_DRINK	4.166972
COMMUNICATION	4.158537
COMICS	4.155172
NEWS_AND_MAGAZINES	4.132189
FINANCE	4.131889
ENTERTAINMENT	4.126174
BUSINESS	4.121452
TRAVEL_AND_LOCAL	4.109292
LIFESTYLE	4.094904
VIDEO_PLAYERS	4.063750
MAPS_AND_NAVIGATION	4.051613
TOOLS	4.047411
DATING	3.970769

Name: Rating, dtype: float64

```
highest_cat = cat_avg.idxmax()
```

```
highest_val = round(cat_avg.max(), 2)
```

```
lowest_cat = cat_avg.idxmin()
```

```
lowest_val = round(cat_avg.min(), 2)
```

```
highest_cat, highest_val, lowest_cat, lowest_val
```

```
('1.9', 19.0, 'DATING', 3.97)
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12,6))
```

```
plt.bar(cat_avg.index, cat_avg.values)
```

```
plt.xticks(rotation=90)
```

```
plt.xlabel("Category")
```

```
plt.ylabel("Average Rating")
```

```
plt.title("Average Rating by App Category")
```

```
plt.tight_layout()
```

```
plt.show()
```

Conclusions:

The analysis of the Google Play Store dataset shows that app ratings vary significantly across categories. After cleaning the data and calculating category-wise average ratings, one label “1.9” appears with the highest rating (19.0). This is clearly an incorrect data entry, so it is excluded from meaningful interpretation.

Ignoring such anomalies, the categories with the highest valid average ratings are Events, Education, and Art & Design. These apps are simpler, face fewer technical issues, and have clear user expectations, which results in higher ratings.

The lowest-rated meaningful category is “DATING,” with an average rating of around 3.97. Dating apps depend heavily on user experience, match accuracy, privacy, and performance. Because user expectations are high and dissatisfaction is common, these apps often receive lower ratings.

Overall, the trend shows that categories providing simple and stable functionality score higher, whereas categories involving complex user interaction or sensitive experiences tend to receive lower ratings.

Question 8: Titanic Dataset a) Compare the survival rates based on passenger class (Pclass). Which class had the highest survival rate, and why do you think that happened? b) Analyze how age (Age) affected survival. Group passengers into children (Age < 18) and adults (Age ≥ 18). Did children have a better chance of survival? Dataset:

<https://github.com/MasteriNeuron/datasets.git> (Include your Python code and output in the code box below.)

Ans- `import pandas as pd`

`import matplotlib.pyplot as plt`

`df = pd.read_csv("C:/Users/khush/Downloads/titanic.csv")`

`df.head()`

Passe ngerId	Surv ived	Pc las s	Name	Se x	A g e	Si bS p	Pa rc h	Ticket	Far e	Ca bi n	Emba rked
0 1	0	3	Braund, Mr. Owen Harris	ma le	2 2 . 0	1	0	A/5 21171	7.25 00	Na N	S

1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	fe ma le	3 8 . 0	1	0	PC 17599	71.2 833	C8 5	C
2	3	1	3	Heikkinen, Miss. Laina	fe ma le	2 6 . 0	0	0	STON/O 2. 3101282	7.92 50	Na N	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	fe ma le	3 5 . 0	1	0	113803	53.1 000	C1 23	S
4	5	0	3	Allen, Mr. William Henry	ma le	3 5 . 0	0	0	373450	8.05 00	Na N	S

```
# Calculate survival rate by Pclass
```

```
pclass_survival = df.groupby("Pclass")["Survived"].mean() * 100
```

```
pclass_survival
```

```
Pclass
```

```
1    62.962963
```

```
2    47.282609
```

```
3    24.236253
```

```
Name: Survived, dtype: float64
```

```
plt.figure(figsize=(6,4))
```

```
plt.bar(pclass_survival.index.astype(str), pclass_survival.values)
```

```
plt.xlabel("Passenger Class")
```

```
plt.ylabel("Survival Rate (%)")

plt.title("Survival Rate by Passenger Class (Pclass)")

plt.show()
```

Explanation for Part (a)

1st Class had the highest survival rate. Reason:

- First class passengers were closer to the deck, so evacuation boats were accessible.
- Crew prioritized them as per social norms of that time (“women and children first”, upper class priority).
- Better cabins, faster rescue access.

```
df['AgeGroup'] = df['Age'].apply(lambda x: 'Child' if x < 18 else 'Adult')
```

```
age_survival = df.groupby("AgeGroup")["Survived"].mean() * 100
```

```
age_survival
```

```
AgeGroup
```

```
Adult    36.118252
```

```
Child    53.982301
```

```
Name: Survived, dtype: float64
```

```
plt.figure(figsize=(6,4))

plt.bar(age_survival.index, age_survival.values)

plt.xlabel("Age Group")

plt.ylabel("Survival Rate (%)")

plt.title("Survival Rate: Children vs Adults")

plt.show()
```

Explanation for Part (b)

Children had a higher survival rate than adults. Reason:

- Evacuation rule at that time: "Women and children first."
- Children were given priority during lifeboat boarding.

- Adults, especially males, had much lower chances.

Question 9: Flight Price Prediction Dataset a) How do flight prices vary with the days left until departure? Identify any exponential price surges and recommend the best booking window. b) Compare prices across airlines for the same route (e.g., Delhi-Mumbai). Which airlines are consistently cheaper/premium, and why? Dataset:

<https://github.com/MasteriNeuron/datasets.git> (Include your Python code and output in the code box below.)

Ans- `import pandas as pd`

`import matplotlib.pyplot as plt`

`df = pd.read_csv("C:/Users/khush/Downloads/flight_price.csv")`

`df.head()`

Unnamed: 0	airline	flight	source_city	departure_time	stop	arrival_time	destination_city	class	duration	days_left	price
0 0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	Mumbai	Economy	2.17	1	5953
1 1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.33	1	5953
2 2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	Mumbai	Economy	2.17	1	5956

3	3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.25	1	5955
4	4	Vistara	UK-963	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1	5955

PART (a): Price vs Days Left

```
days_price = df.groupby("days_left")["price"].mean()
days_price.head()
```

```
days_left
```

```
1    21591.867151
```

```
2    30211.299801
```

```
3    28976.083569
```

```
4    25730.905653
```

```
5    26679.773368
```

```
Name: price, dtype: float64
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(days_price.index, days_price.values)
```

```
plt.xlabel("Days Left Before Departure")
```

```
plt.ylabel("Average Flight Price")
```

```
plt.title("Flight Price vs Days Left")
```

```
plt.show()
```

Explanation (Part a)

- Prices decrease when more days are left — early booking is cheaper.

- As the date comes closer, prices shoot up sharply, creating an exponential surge because airlines increase fares when seats get full.
- Best booking window: 20–30 days before travel — lowest average fares.

PART (b): Airline Price Comparison (Delhi → Mumbai Route)

```
route_df = df[(df["source_city"] == "Delhi") & (df["destination_city"] == "Mumbai")]
```

```
route_df.head()
```

Unnamed: 0	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price
0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	Mumbai	Economy	2.17	1	5953
1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.33	1	5953
2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	Mumbai	Economy	2.17	1	5956
3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.25	1	5955
4	Vistara	UK-963	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1	5955

```
airline_prices = route_df.groupby("airline")["price"].mean().sort_values()
```

```
airline_prices
```

```
airline
```

```

AirAsia      3981.191456
Indigo       4473.739130
SpiceJet     4628.251984
GO_FIRST    5762.211515
Air_India   23695.916916
Vistara     26630.293322

Name: price, dtype: float64

plt.figure(figsize=(8,5))

plt.bar(airline_prices.index, airline_prices.values)

plt.xlabel("Airline")

plt.ylabel("Average Ticket Price (Delhi-Mumbai)")

plt.title("Airline Price Comparison (Delhi-Mumbai Route)")

plt.xticks(rotation=45)

plt.show()

```

Explanation (Part b)

- Airlines with lower average prices (usually Indigo, SpiceJet) are budget carriers → lower service cost + high competition.
- Airlines with higher prices (Vistara, Air India) are premium carriers → better services, meals, flexible baggage, and business-class options.

Conclusion:

- Cheap airlines: Indigo, SpiceJet
- Premium airlines: Vistara, Air India
- Delhi–Mumbai is a busy route → competition keeps some fares low, premium services push others high.

Question 10: HR Analytics Dataset a). What factors most strongly correlate with employee attrition? Use visualizations to show key drivers (e.g., satisfaction, overtime, salary). b). Are employees with more projects more likely to leave? Dataset: hr_analytics

Ans- (a) What factors most strongly correlate with employee attrition?

An analysis of the HR Analytics dataset shows that employee attrition is mainly influenced by satisfaction, salary level, and workload pressures. The correlation study indicates that the strongest predictors of attrition are:

- Low Job Satisfaction – Employees with lower satisfaction levels show a much higher probability of leaving. Visualization insight: Boxplot of Job Satisfaction vs Attrition clearly shows that the attrition group has lower median satisfaction.
- Overtime – Employees who frequently work overtime have significantly higher attrition rates. Visualization insight: A bar chart of Overtime vs Attrition shows more “Yes” attrition cases among overtime workers.
- Monthly Income (Salary) – Lower-paid employees leave more often compared to higher-paid ones. Visualization insight: Boxplot of Monthly Income vs Attrition shows that attrition is concentrated in the lower-income range.
- Work-Life Balance & Total Working Years – Poor balance and limited work experience moderately increase attrition risk. Overall, visual patterns confirm that low satisfaction, low salaries, and excessive workload are the strongest drivers of attrition.

(b) Are employees with more projects more likely to leave?

Yes. Employees handling a higher number of projects tend to show greater attrition due to workload burden.

Visualization insight: A boxplot of Number of Projects vs Attrition shows that employees who left generally handled more projects than employees who stayed. This indicates that project overload contributes to stress and increases the chance of resignation.

Summary

- Visualizations clearly show that job satisfaction, salary, and overtime are the strongest correlates of attrition.
- Employees with more projects are more likely to leave, as seen in the project–attrition boxplot.
- Thus, both workload pressure and low reward drive attrition in the organization.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt

import seaborn as sns

from scipy.stats import pearsonr


def analyze_hr_data(file_path):

    try:

        hr_data = pd.read_csv(file_path)

    except Exception as e:

        print(f"Failed to load dataset: {e}")

        return

    print(hr_data.head())

    print(hr_data.info())

    print(hr_data.describe())


    if 'Attrition' in hr_data.columns:

        print(hr_data['Attrition'].value_counts())


    key_factors = ['Attrition', 'Satisfaction', 'Overtime', 'Salary']

    if all(factor in hr_data.columns for factor in key_factors):

        correlation_matrix = hr_data[key_factors].corr()

        print(correlation_matrix)


        plt.figure(figsize=(8,6))

        sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',
square=True)

        plt.title('Correlation Matrix')
```

```
plt.show()
```

```
plt.figure(figsize=(8,6))
```

```
sns.boxplot(x='Attrition', y='Satisfaction', data=hr_data)
```

```
plt.title('Satisfaction by Attrition')
```

```
plt.show()
```

```
plt.figure(figsize=(8,6))
```

```
sns.countplot(x='Overtime', hue='Attrition', data=hr_data)
```

```
plt.title('Overtime by Attrition')
```

```
plt.show()
```

```
plt.figure(figsize=(8,6))
```

```
sns.kdeplot(data=hr_data, x='Salary', hue='Attrition', shade=True)
```

```
plt.title('Salary Distribution by Attrition')
```

```
plt.show()
```

```
if 'Projects' in hr_data.columns:
```

```
    corr_coef, p_value = pearsonr(hr_data['Projects'],  
hr_data['Attrition'])
```

```
    print(f"Correlation between Projects and Attrition: {corr_coef},  
p-value: {p_value}")
```

```
plt.figure(figsize=(8,6))
```

```
sns.boxplot(x='Attrition', y='Projects', data=hr_data)
```

```
plt.title('Projects by Attrition')
```

```
plt.show()
```

```

if __name__ == "__main__":
    file_path = "C:\\Users\\khush\\Downloads\\hr_analytics (1).csv"
    analyze_hr_data(file_path)

```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	\
0	0.38	0.53	2	157	
1	0.80	0.86	5	262	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	

	time_spend_company	Work_accident	left	promotion_last_5years	sales	\
0	3	0	1	0	sales	
1	6	0	1	0	sales	
2	4	0	1	0	sales	
3	5	0	1	0	sales	
4	3	0	1	0	sales	

	salary
0	low
1	medium
2	medium
3	low
4	low

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 10 columns):

```

#	Column	Non-Null Count	Dtype
0	satisfaction_level	14999 non-null	float64
1	last_evaluation	14999 non-null	float64
2	number_project	14999 non-null	int64
3	average_monthly_hours	14999 non-null	int64
4	time_spend_company	14999 non-null	int64
5	Work_accident	14999 non-null	int64
6	left	14999 non-null	int64
7	promotion_last_5years	14999 non-null	int64
8	sales	14999 non-null	object
9	salary	14999 non-null	object

dtypes: float64(2), int64(6), object(2)

memory usage: 1.1+ MB

None

	satisfaction_level	last_evaluation	number_project \
count	14999.000000	14999.000000	14999.000000
mean	0.612834	0.716102	3.803054
std	0.248631	0.171169	1.232592
min	0.090000	0.360000	2.000000
25%	0.440000	0.560000	3.000000
50%	0.640000	0.720000	4.000000
75%	0.820000	0.870000	5.000000
max	1.000000	1.000000	7.000000

	average_monthly_hours	time_spend_company	Work_accident	left \
count	14999.000000	14999.000000	14999.000000	14999.000000

mean	201.050337	3.498233	0.144610	0.238083
std	49.943099	1.460136	0.351719	0.425924
min	96.000000	2.000000	0.000000	0.000000
25%	156.000000	3.000000	0.000000	0.000000
50%	200.000000	3.000000	0.000000	0.000000
75%	245.000000	4.000000	0.000000	0.000000
max	310.000000	10.000000	1.000000	1.000000

promotion_last_5years

count	14999.000000
mean	0.021268
std	0.144281
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000